

M1 - GPGPU

TP3 - Calcul d'histogramme

Maxime MARIA, Vincent LARROQUE

2024-2025

Au cours de ce TP, nous allons étudier différentes implémentations d'un calcul d'histogramme à travers leur performance. Pour cela, le code source produit sera accompagné d'un rapport présentant les performances obtenues avec les stratégies envisagées et des commentaires associés.

Au cours de ce TP, vous allez répétez certaines tâches, notamment des benchmarks. La production de graphique ainsi qu'un code factorisé sont bienvenues afin de faciliter l'implémentation et la lecture des performances mesurées.

Rappel : Un histogramme est une représentation qui, pour un tableau donné, donne le nombre d'occurrences de chaque valeur le composant.

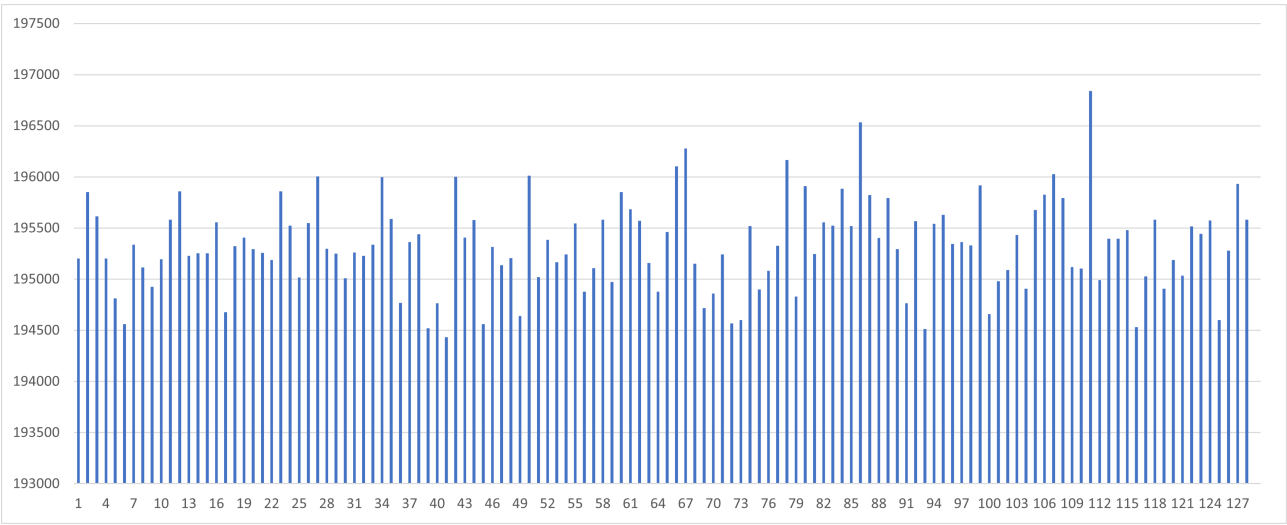


FIGURE 1 – Histogramme obtenu à partir d'un tableau de 100 000 000 éléments contenant des valeurs aléatoires d'une distribution uniforme $p \in [0, 128[$.

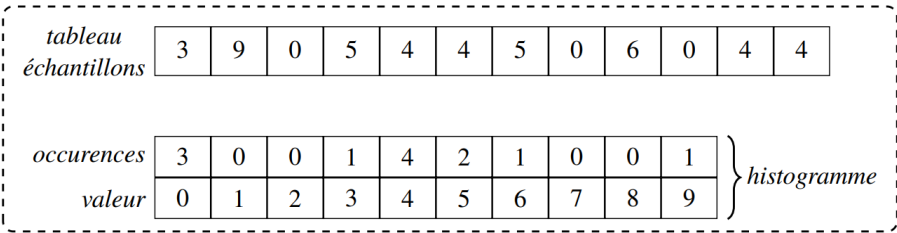


FIGURE 2 – Représentation d'un tableau et de son histogramme tout deux sous forme de tableau.

1 Exercice 1 - Implémentation naïve

Dans cet exercice, nous allons calculer l'histogramme d'un vecteur aléatoire donné en répartissant les tâches aux threads GPU dont nous disposons.

1. Utilisez la classe `Generator` pour générer un tableau aléatoire d'une taille de 100 000 000 éléments à partir d'une distribution comprise entre 0 et 128.
2. Réalisez le code nécessaire au calcul de l'histogramme correspondant sur CPU, il va nous permettre de valider le code GPU que l'on va rédiger dans la suite de l'exercice.
3. *Bonus : Si vous ordinateur a plus de 8Go de RAM, testez votre programme avec un tableau aléatoire d'une taille de 2 147 483 648 et une distribution de 1, fonctionne-t-il toujours ?*
4. Nous allons maintenant réaliser le code nécessaire au calcul de l'histogramme correspondant sur GPU, pour cela vous devrez utiliser une opération atomique vue en cours.

Attention : Au lieu d'assigner une seule valeur du tableau à chacun des threads, on va plutôt leur donner un intervalle à traiter. Votre implémentation doit donc utiliser un thread pour calculer l'histogramme de N valeurs. N doit être configurable globalement.

Vérifiez que les valeurs calculées sur le GPU sont bien les mêmes que celles du CPU.

5. Réalisez des tests de performance avec les outils proposés par CUDA (`nv-nsight-cu-cli` / `ncu` ou `nvprof --print-gpu-trace` dans le cas d'une carte graphique plus ancienne).

Effectuez les tests de performance pour les valeurs :

$$N = \{1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024\}$$

Quelle est la meilleure valeur ? Commentez les performances obtenues.

2 Exercice 2 - Vers une meilleure implémentation

Les performances obtenues à la fin de l'exercice précédent semblent améliorables. En effet, les nombreuses opérations atomiques appliquées sur la mémoire vive de la carte forcent une synchronisation à haut-niveau ralentissant l'exécution du kernel.

Rappel : Plusieurs types de mémoires sont disponibles en CUDA qui comporte chacun des avantages et des inconvénients. Jusqu'ici, nous nous sommes servis de la mémoire globale (à partir de `cudaMalloc`) qui permet un accès à la totalité de la mémoire disponible dans la VRAM et son utilisation par tous les threads des kernels à qui on la transmet. Elle est cependant moins performante que la mémoire partagée ou la mémoire constante qui ont des capacités et/ou accès plus restreints. En effet, la mémoire partagée notamment n'est partagée qu'entre des threads d'un même bloc.

Afin de conserver une base de comparaison cohérente, les étapes suivantes devront être testées dans les mêmes conditions que notre première implémentation.

1. Nous allons tenter de réduire la portée des synchronisations induites par nos opérations atomiques.
Implémentez un second kernel dans lequel chaque bloc va travailler sur une portion de taille N du vecteur de données et réaliser son histogramme en mémoire partagée. Il devra ensuite reporter ses résultats dans l'histogramme global.
Réalisez les mêmes tests (avec les mêmes valeurs de N) que pour notre première implémentation et commentez les performances obtenues.
2. *Bonus : Si vous ne l'avez pas déjà fait, réalisez un kernel identique tout en essayant d'améliorer la cohérence des accès à la mémoire. Réalisez les tests et commentez les performances obtenues.*
3. Les tests précédents ont permis d'obtenir des performances satisfaisantes tout en conservant un parallélisme similaire. Il peut cependant être utile d'essayer des répartitions des tâches différentes qui peuvent permettre de mieux satisfaire les architectures visées.

Comme pour les questions précédentes, réalisez un kernel travaillant sur un intervalle de taille N du tableau. Dans celui-ci, calculez l'histogramme local en mémoire partagée en assignant un thread à une seule valeur de l'histogramme puis reportez les résultats en mémoire globale. Réalisez les mêmes tests (avec les mêmes valeurs de N) que pour notre première implémentation et commentez les performances obtenues.